

Application Layer

STANDARD CLIENT-SERVER APPLICATIONS

During the evolution of the Internet, several **client–server applications** have been developed to provide different services. Understanding these applications helps in designing customized network applications. The major applications include:

- **HTTP/WWW** – Accessing web pages and websites.
- **FTP** – Transferring files between computers.
- **E-mail** – Sending and receiving electronic messages.
- **TELNET** – Remote login to another computer.
- **SSH** – Secure remote access and login.
- **DNS** – Mapping domain names to IP addresses.

World Wide Web (WWW)

- The **World Wide Web (WWW)** was proposed by Tim Berners-Lee in **1989** at CERN to facilitate the **sharing of research information among scientists**.
- The commercial use of the Web began in the **early 1990s**.
- The Web is a **global repository of information** where documents, called **web pages**, are stored on servers located around the world.
- The popularity of the Web is due to two important characteristics:
 - **Distributed Nature:** Web pages are stored on different servers worldwide, allowing unlimited growth.
 - **Linked Structure:** Web pages can be connected through hyperlinks, enabling easy navigation between related documents.

World Wide Web (WWW)

- The concept of linking documents originated from **Hypertext**, which allows users to move from one document to another by clicking a link.
- Today, the term **Hypermedia** is used because web pages can contain:
 - Text
 - Images
 - Audio
 - Video
 - Animations

Architecture of the World Wide Web (WWW)

The **World Wide Web (WWW)** is a **distributed client-server system** that allows users to access information over the Internet.

1. Client

- The client is the user's computer or device.
- It uses a **web browser** (such as Chrome, Firefox, or Edge) to request and view web pages.

2. Server

- A server stores web pages and provides them to clients upon request.

3. Website (Site)

- Information on the Web is distributed across many locations called **sites**.
- Each site contains one or more web pages.

Architecture of the World Wide Web (WWW)

4. Web Pages

- A web page is a file with a unique **name and address (URL)**.
- Web pages can be:
 - **Simple Web Page:** Contains only information and no links to other pages.
 - **Composite Web Page:** Contains one or more links (hyperlinks) to other web pages within the same site or different sites.

5. Hyperlinks

- Hyperlinks connect web pages together.
- They allow users to navigate from one page to another.

Working of WWW Architecture

1. The user enters a URL in a web browser.
2. The browser (client) sends a request to the web server.
3. The server locates the requested web page.
4. The server sends the page back to the browser.
5. The browser displays the page, including any hyperlinks to other pages.

Retrieval of Web Documents, Web Client, and Web Server

Retrieval of Web Documents

- ▶ A web document may contain references (hyperlinks) to other files such as text documents, images, videos, or web pages.

Example

- ▶ Consider a scientific document:
 - **File A** – Main document stored in Site I.
 - **File B** – Large image stored in Site I.
 - **File C** – Referenced text file stored in Site II.

Retrieval of Web Documents, Web Client, and Web Server

Transactions Required

► To view the complete document, **three separate transactions** are needed:

1. First Transaction

- Retrieves the main document (File A).
- File A contains references (links) to File B and File C.

2. Second Transaction

- Triggered when the user clicks the image reference.
- Retrieves the image file (File B).

3. Third Transaction

- Triggered when the user clicks the text-file reference.
- Retrieves the text file (File C).

Retrieval of Web Documents, Web Client, and Web Server

- File A, File B, and File C are **independent web pages/files**.
- Each file has its own **name and address (URL)**.
- Even if File B and File C are referenced in File A, they can be accessed independently.
- Retrieving one file does not automatically retrieve the others.

Web Client (Browser)

- ▶ A **Web Client** or **Browser** is software used to access and display web pages.

Architecture of a Browser

- ▶ A browser consists of three main components:

1. Controller

- Receives input from the user through the keyboard or mouse.
- Controls access to web documents.

Web Client (Browser)

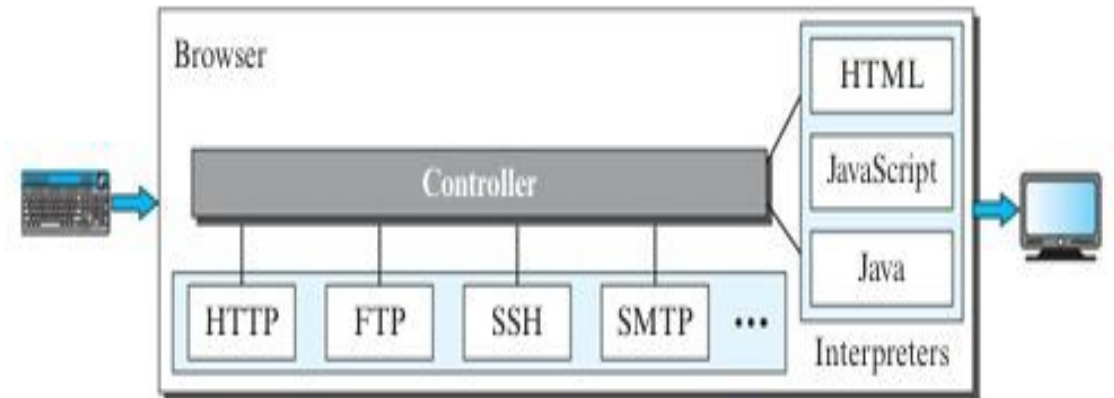
2. Client Protocols

- Used to communicate with web servers.
- Examples:
 - HTTP (HyperText Transfer Protocol)
 - FTP (File Transfer Protocol)

3. Interpreters

- Interpret and display the contents of web pages.
- Examples:
 - HTML Interpreter
 - Java Interpreter
 - JavaScript Interpreter

Figure 2.9 Browser



Web Client (Browser)

► Working of a Browser

1. User enters a URL or clicks a link.
2. Controller sends a request using a client protocol.
3. Requested document is retrieved.
4. Interpreter processes the document.
5. Document is displayed on the screen.

► Examples of Browsers

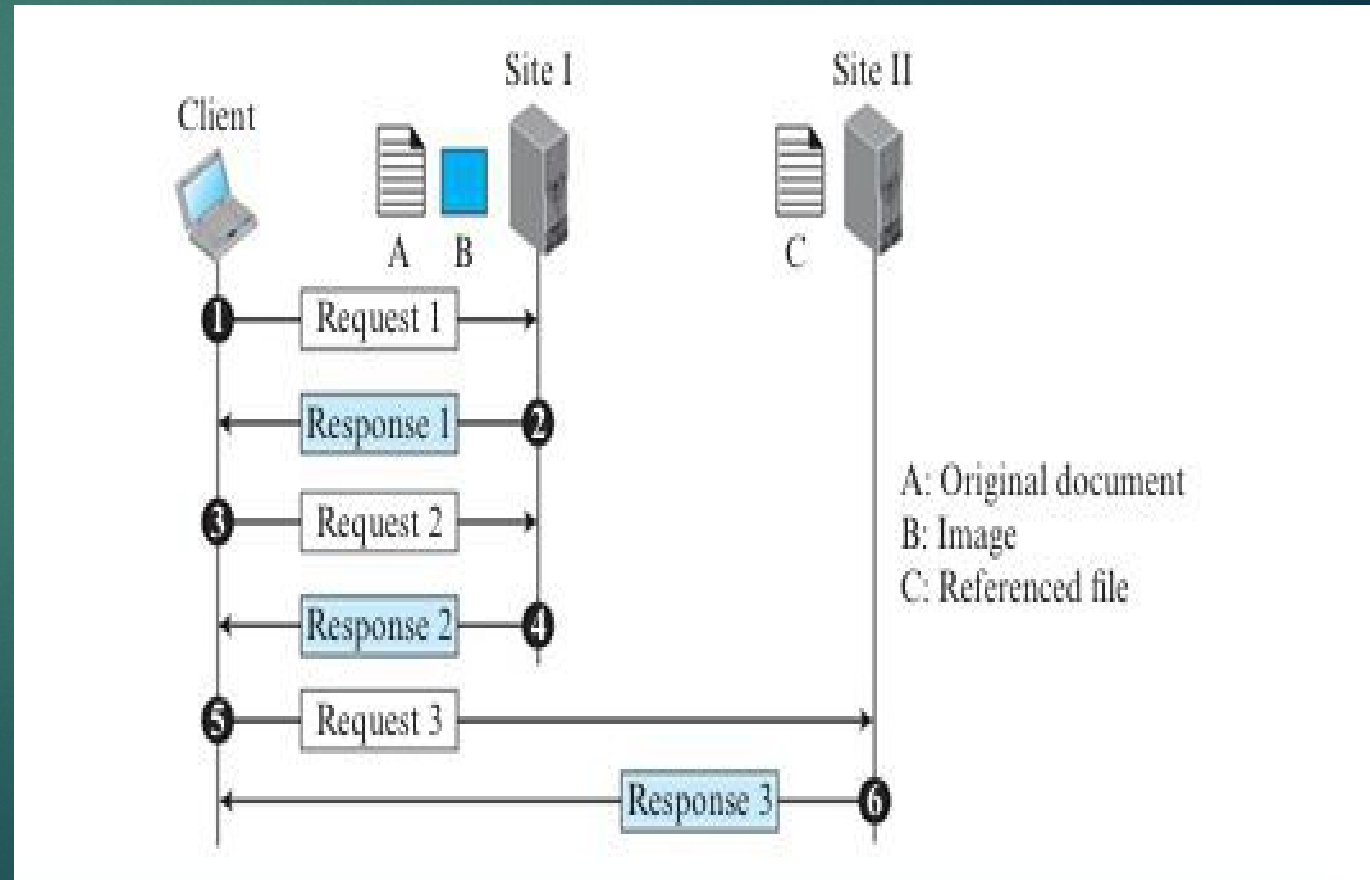
- Internet Explorer
- Netscape Navigator
- Firefox
- Google Chrome

Web Server

- ▶ A **Web Server** stores web pages and delivers them to clients when requested.

- ▶ **Functions of a Web Server**

- Stores web pages and files.
- Receives requests from browsers.
- Sends requested documents to clients.



Web Server

Examples of Web Servers

- ❖ Apache Server
- ❖ Microsoft Internet Information Server (IIS)

Uniform Resource Locator (URL)

- ▶ A **Uniform Resource Locator (URL)** is the unique address used to identify and locate a web page or resource on the Internet. To access a web page, four pieces of information are required:

1. **Protocol**
2. **Host**
3. **Port**
4. **Path**

Components of a URL

1. Protocol

- Specifies the client-server application used to access the resource.
- Indicates how data is transferred between the browser and the server.

► Examples:

- HTTP (HyperText Transfer Protocol)
- HTTPS (HyperText Transfer Protocol Secure)
- FTP (File Transfer Protocol)

Components of a URL

2. Host

- Identifies the server where the resource is stored.
- Can be:
 - An IP address
 - A Domain Name

Examples:

- ▶ 64.23.56.17
- ▶ forouzan.com

Components of a URL

3. Port

- A **16-bit integer** that identifies a specific service on the server.
- Usually predefined for common protocols.
- ▶ **Examples:**
 - HTTP → Port 80
 - HTTPS → Port 443
 - FTP → Port 21
- ▶ If the default port is used, it is usually omitted from the URL.
- ▶ **Example::80**

Components of a URL

4. Path

- Specifies the location and name of the file on the server.
- Consists of directories followed by the file name.

▶ UNIX Example:

▶ /top/next/last/myfile

▶ Here:

- top → top-level directory
- next → subdirectory
- last → subdirectory
- myfile → file name

URL Format

- ▶ **Without Port Number (Most Common)**
- ▶ protocol://host/path
- ▶ **Example:**
- ▶ <http://www.forouzan.com/books/index.html>

URL Format

- ▶ **With Port Number**
 - ▶ protocol://host:port/path
-
- ▶ **Example:**
 - ▶ `http://www.forouzan.com:80/books/index.html`

URL Format

- URL stands for **Uniform Resource Locator**.
- It uniquely identifies a web page or resource on the Internet.
- A URL consists of **Protocol, Host, Port, and Path**.
- The **host** identifies the server.
- The **port** identifies the service running on the server.
- The **path** identifies the exact file or resource.
- The **protocol** specifies how the resource is accessed.

Web Documents

- ▶ Web documents are classified into three types:

1. Static Documents

- Fixed content documents stored on a web server.
 - Users can only view a copy of the document.
 - Content remains the same unless updated by the server administrator.
 - Created using languages such as **HTML, XML, XSL, and XHTML**.
- ▶ **Example:** A college website's information page.

Web Documents

2. Dynamic Documents

- Created by the server when requested by a browser.
 - Content may change for each request.
 - Generated using programs or scripts such as **JSP, ASP, and ColdFusion**.
- ▶ **Example:** Current date and time displayed on a webpage.

Web Documents

3. Active Documents

- Contain programs or scripts that run on the client (browser) side.
 - Used for animations, interactive forms, and user interactions.
 - Created using **Java Applets** or **JavaScript**.
- **Example:** Interactive games or animated graphics on a webpage.

HyperText Transfer Protocol (HTTP)

Definition

► **HTTP (HyperText Transfer Protocol)** is the protocol used to transfer web pages between a web browser (client) and a web server.

- The **client** sends an HTTP request.
- The **server** sends an HTTP response.
- HTTP uses **TCP** as its transport protocol.
- **Server Port Number:** 80
- **Client Port Number:** Temporary (ephemeral) port.

HyperText Transfer Protocol (HTTP)

Features of HTTP

- Client-Server protocol.
- Uses TCP for reliable communication.
- Requires a TCP connection before data transfer.
- Connection is terminated after communication.

Non Persistent and Persistent Connections

- ▶ When a web page contains multiple objects (images, videos, etc.), HTTP can retrieve them using:

1. Non Persistent Connection

- A separate TCP connection is created for each request/response.
- Used in **HTTP/1.0**.

▶ Steps

1. Client opens a TCP connection and sends a request.
2. Server sends the response and closes the connection.
3. Client receives the data and closes the connection.

Non Persistent and Persistent Connections

Example

- ▶ If a webpage contains **N images**, then:
 - 1 connection for the HTML page
 - N connections for N images
- ▶ **Total Connections = $N + 1$**

Non Persistent and Persistent Connections

► Disadvantages

- High overhead.
- More connection setup and termination time.
- Increased server workload.
- Requires more buffers and resources.

Non Persistent and Persistent Connections

2. Persistent Connection

- A single TCP connection is used for multiple requests and responses.
- Default in **HTTP/1.1**.

► Advantages

- Reduced overhead.
- Faster page loading.
- Less network traffic.
- Better server performance.
- The connection is closed:
 - At the client's request, or
 - When a timeout occurs

Non Persistent and Persistent Connections

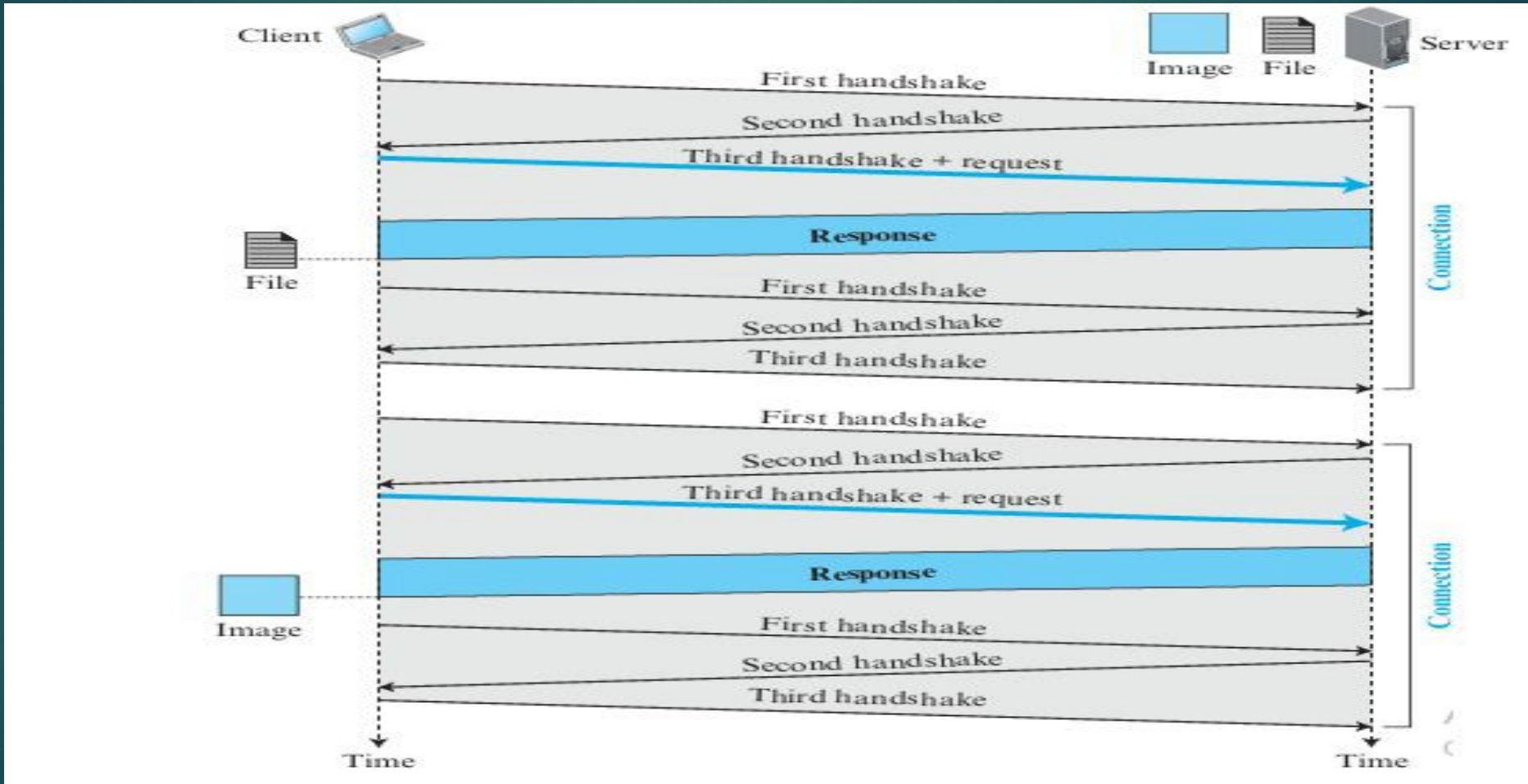
► Nonpersistent Connection (Example)

- A webpage contains a text file with one linked image.
- Both files are stored on the same server.
- **Two separate TCP connections** are required:
 - One for the text file.
 - One for the image file.
- Each connection requires:
 - Connection establishment (TCP handshake).
 - Data transfer.
 - Connection termination.

Non Persistent and Persistent Connections

- Multiple objects require multiple connections.
- Increased **round-trip time (RTT)** due to repeated handshakes.
- Additional buffers and variables must be allocated for each connection.
- Creates higher overhead, especially on the server.

Non Persistent and Persistent Connections



Non Persistent and Persistent Connections

► Persistent Connection

- Default connection type in **HTTP/1.1**.
- A single TCP connection is used for multiple requests and responses.
- The server keeps the connection open after sending a response.
- The connection is closed:
 - At the client's request, or
 - When a timeout occurs.

Non Persistent and Persistent Connections

- The sender usually specifies the length of the data being sent.
- For dynamically generated documents, if the length is unknown, the server closes the connection after transmission to indicate the end of data.
- Saves time and network resources.
- Requires only one set of buffers and variables at each site.
- Eliminates repeated connection establishment and termination.
- Reduces RTT and improves overall performance.

HTTP Request Message

► An HTTP request message consists of four parts:

1. Request Line
2. Header Lines
3. Blank Line
4. Body (Optional)

HTTP Request Message

- ▶ **1. Request Line**

- ▶ The request line contains three fields separated by a space (SP) and terminated by CRLF (Carriage Return and Line Feed).

- ▶ **Method**

- ▶ Specifies the type of request sent by the client.

HTTP Request Message

Table 2.1 *Methods*

<i>Method</i>	<i>Action</i>
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

HTTP Request Message

URL

- Specifies the address and name of the requested web page.
- Identifies the resource to be accessed.

▶ **Example:**

▶ /index.html

Version

- Specifies the HTTP protocol version.
- Current version: **HTTP/1.1**

▶ **Example:**

▶ HTTP/1.1

HTTP Request Message

2. Request Header Lines

- Zero or more header lines may follow the request line.
- Provide additional information from the client to the server.
- Allow the client to specify preferences and conditions.

▶ Format

▶ Header-Name: Value

▶ Examples

▶ Host: www.example.com Accept: text/html

User-Agent: Chrome

If-Modified-Since: Mon, 10 Jun 2025

HTTP Request Message

3. Blank Line

- A blank line separates the header section from the body section.
- Indicates the end of the headers.

4. Body (Optional)

- Present only in some request messages.
- Commonly used with **PUT** and **POST** methods.
- Contains:
 - Comments
 - Form data
 - Files to be uploaded
 - Information used to modify a web page

HTTP Response Message

► An HTTP response message is sent by the **server to the client**. It consists of four parts:

1. **Status Line**
2. **Header Lines**
3. **Blank Line**
4. **Body (Optional)**

HTTP Response Message

1. Status Line

- ▶ The first line of a response message is called the **Status Line**.
- ▶ **Format**
- ▶ Version SP Status-Code SP Status-Phrase CRLF
- ▶ **Fields**
- ▶ **Version**
 - Specifies the HTTP protocol version used by the server.
 - Current version: **HTTP/1.1**
- ▶ **Example:**
- ▶ HTTP/1.1

HTTP Response Message

Status Code Range / Code

100–199

200–299

300–399

400–499

500–599

200

301

302

404

500

Meaning

Informational messages

Successful request

Redirection to another URL

Client-side error

Server-side error

OK

Moved Permanently

Found (Temporary Redirect)

Not Found

Internal Server Error

HTTP Response Message

Status Phrase

- ▶ Text explanation of the status code.
- ▶ **Examples:**
- ▶ 200 OK
- ▶ 404 Not Found
- ▶ 500 Internal Server Error

HTTP Response Message

2. Response Header Lines

- Zero or more header lines may follow the status line.
- Provide additional information from the server to the client.
- Describe the document or server.

▶ **Format**

▶ Header-Name: Value

HTTP Response Message

3. Blank Line

- ↴ Separates the header section from the body section.
- ↴ Marks the end of the header lines.

4. Body (Optional)

- ↴ Contains the actual document or data requested by the client.
- ↴ May include:
 - ↵ HTML pages
 - ↵ Images
 - ↵ Audio files
 - ↵ Video files
 - ↵ Error messages

HTTP Request and Response Headers

Figure 2.12 *Formats of the request and response messages*

